

Clase 115 — Secure coding y defensa de aplicaciones web

Parte: **4 — Seguridad de aplicaciones web** · Fuente: *OWASP ASVS / OWASP Cheat Sheet Series* 
Duración estimada: **120 min** · Nivel: **Avanzado**

Objetivo

Cerrar la parte pasando del ataque a la **defensa**: cómo escribir código seguro y arquitecturas resistentes que eliminen de raíz las vulnerabilidades vistas. Aprenderás los principios de secure coding, las defensas concretas por categoría OWASP y cómo integrarlas en el ciclo de desarrollo (DevSecOps).

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Aplicar** principios de diseño seguro (defensa en profundidad, mínimo privilegio).
2. **Implementar** las defensas correctas por categoría (inyección, XSS, authz).
3. **Usar** cabeceras de seguridad (CSP, HSTS, cookies seguras) con criterio.
4. **Integrar** SAST/DAST/SCA y OWASP ASVS en el SDLC.
5. **Evaluar** una aplicación contra un checklist de secure coding.

Temas

#	Tema	Por qué importa
1	Principios de diseño seguro	Base transversal
2	Defensas por categoría OWASP	Corregir la causa raíz
3	Validación y codificación de salida	Contra inyección/XSS
4	Cabeceras de seguridad	Endurecimiento del cliente
5	Gestión segura de secretos y dependencias	A02, A06
6	SAST/DAST/SCA en CI/CD	Automatizar la seguridad
7	OWASP ASVS como checklist	Verificación estructurada

Definiciones y características

- **Secure coding**: prácticas de programación que previenen vulnerabilidades. Característica: es más barato prevenir que parchear.
- **Defensa en profundidad**: múltiples capas de control. Característica: si una falla, otra contiene el daño.

- **Mínimo privilegio**: cada componente solo con los permisos necesarios. Característica: limita el impacto de un compromiso.
- **CSP (Content Security Policy)**: cabecera que restringe recursos ejecutables. Característica: mitiga XSS aunque exista el bug.
- **ASVS**: estándar de verificación de OWASP con requisitos por nivel. Característica: convierte "ser seguro" en una checklist auditable.
- **SAST/DAST/SCA**: análisis estático, dinámico y de composición. Característica: automatizan la detección en el pipeline.

Herramientas y preparación

- **OWASP ASVS** y **OWASP Cheat Sheet Series**.
- **Semgrep** (SAST), **OWASP ZAP** (DAST), **Dependabot/Trivy/OWASP Dependency-Check** (SCA).
- **securityheaders.com** y **CSP Evaluator** para revisar cabeceras.

```
# SAST rápido con Semgrep
pipx install semgrep
semgrep --config=auto ./tu-proyecto
```

Laboratorio guiado

Ejercicio aplicado de defensa (revisión y corrección de código).

1. Toma un endpoint vulnerable a SQLi de clases previas y **reescribelo** con consultas parametrizadas.
2. Corrige un XSS aplicando **codificación de salida por contexto** y añade una CSP restrictiva.
3. Endurece las cookies: `HttpOnly`, `Secure`, `SameSite`, y verifica con DevTools.
4. Añade cabeceras de seguridad (`HSTS`, `X-Content-Type-Options`, `CSP`) y valida en `securityheaders.com`.
5. Ejecuta **Semgrep** sobre un proyecto y corrige un hallazgo real.
6. Integra un **DAST baseline (ZAP)** y un **SCA** en un pipeline de ejemplo.
7. Audita la app contra un subconjunto de **OWASP ASVS** nivel 1 y documenta gaps.

Ejercicios

1. Reescribe de forma segura una query vulnerable en dos lenguajes.
2. Diseña una CSP para una SPA que solo cargue scripts propios.
3. Configura las cookies de sesión ideales y justifícalo.
4. Corrige un IDOR añadiendo autorización a nivel de objeto.
5. Añade validación de entrada con allowlist en un endpoint.
6. Mapea 10 controles de secure coding a las categorías del OWASP Top 10.

Reto verificable

Toma una aplicación vulnerable (Juice Shop/DVWA o un proyecto propio) y **corrige al menos 3 vulnerabilidades** de categorías distintas, verificando que el ataque original ya no funciona. **Criterio de**

aceptación: entregas el diff/código corregido de 3 fallos (p. ej. SQLi, XSS, IDOR), demuestras que el exploit previo falla tras el cambio, y mapeas cada corrección a su categoría OWASP y requisito ASVS.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Escapar en vez de parametrizar	Frágil; usa prepared statements
CSP con <code>unsafe-inline</code>	Anula la protección; elimina inline y usa nonces
Validación solo en cliente	Inútil como control; valida en servidor
Blocklists en vez de allowlists	Se evaden; prefiere allowlists
Secretos en el repositorio	Fuga; usa gestores de secretos y rota lo expuesto

? Preguntas frecuentes

? **¿Por dónde empiezo a asegurar una app?** Por las categorías de mayor impacto en tu contexto (a menudo access control e inyección) y por un baseline de cabeceras y gestión de sesión.

? **¿SAST o DAST?** Ambos: SAST encuentra fallos en el código, DAST en la app corriendo. Complétalos con SCA para dependencias.

? **¿Qué nivel de ASVS busco?** Nivel 1 como mínimo para cualquier app; niveles 2–3 para aplicaciones sensibles o reguladas.

🔗 Referencias

- OWASP ASVS: <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>
- OWASP Proactive Controls: <https://owasp.org/www-project-proactive-controls/>
- OWASP Top 10 2021: <https://owasp.org/Top10/>

➡ Siguiente clase

Clase 116 - Arquitectura x86/x64 y lenguaje ensamblador